

Exploring Probabilistic Data Structures: Bloom Filters

Bloom filters, conceived by Burton Howard Bloom in 1970, are probabilistic data structures that are used to test whether an element belongs to a set or not.



Processing a large set of entities becomes an expensive and tedious task, particularly for set operations, such as the union and intersection of elements in a particular set. An example would be the processing of a very large set of numbers (e.g., 100 million) and executing a query to determine whether a number belongs to the group or not. The naive method of executing such a query is by iterating over all set members. However, this is expensive, consumes precious CPU cycles and also pollutes the cache memory.

Probabilistic data structures elegantly solve such problems with minimal resources and with an acceptable error rate. They outperform regular solutions like HashMaps and HashTables, which have large memory footprints. In this article, we will discuss one such data structure—the Bloom filter. It is one of the most popular data structures due to its simplicity of implementation, small memory footprint, and efficiency. It is used for request filtering over a large set of data, and used by Web browsers such as Chrome, Web caches of Akamai, and websites like Quora and Medium.

What is a Bloom filter

A Bloom filter was invented by a scientist called Burton Howard Bloom in 1970, to test whether or not an element belongs to a set. The principal property of a Bloom filter is based on probability, in which a positive response of the filter means that the given element could belong to the set. However, a negative response guarantees that the element

is not present in the set. The terminology for the scenario in which an element is present in the set but does not return ‘True’ is termed a false positive or the error rate of the filter.

Defining a Bloom filter

Assume we have an input set $S = [x_1, x_2, \dots, x_n]$ of n elements. A Bloom filter B can be represented by an array of m bits (zero-initialised). The size of Bloom filter is a major factor influencing its error rate. A large size filter usually has a lower error rate.

$$B = [b_1, b_2, \dots, b_{m-1}, b_m]$$

A Bloom filter uses k independent hash functions $h_1, \dots, h_k$ that are assumed to be uniformly distributed over the range $1, \dots, m$.

A standard Bloom filter has two operations defined as follows:

- **add(x)** - Add to the Bloom filter. For

$$1 \leq i \leq k \ B[h_i(x)] = 1 \ \forall x \in S$$

- **(x)** - Check if x is in the Bloom filter. For

$$1 \leq i \leq k \ iB[h_i(x)] = 1 \ \text{return True, else return False.}$$

To represent set S by a Bloom filter B , we need to process each member of S with **add** function. Here is a simple